



Конспект по теме «Репликация и масштабирование. Часть 2»

Содержание:

1. Масштабирование
2. Вертикальное масштабирование (scaling up)
3. Горизонтальное масштабирование (scaling out)
4. Партиционирование и шардинг
5. Примеры реализации в Docker

1. Масштабирование

Масштабирование — это процесс оптимизации системы для работы с растущей нагрузкой. Хорошо спроектированная система изначально учитывает возможность масштабирования, что позволяет избежать дорогостоящей и сложной переделки в будущем.

Существует несколько подходов к масштабированию:

- **Вертикальное (scaling up)** — увеличение мощности текущего сервера.
- **Горизонтальное (scaling out)** — добавление новых серверов в систему.
- **Scaling back** — архивирование части данных для экономии дискового пространства.

2. Вертикальное масштабирование (scaling up)

Это самый простой способ масштабирования. Суть заключается в увеличении мощности существующего сервера путём добавления оперативной памяти, замены процессора или полного переезда на более производительное «железо».

Плюсы вертикального масштабирования:

- не требует изменений в коде приложения;
- управлять одним мощным сервером проще, чем целой системой из нескольких машин.

Минусы вертикального масштабирования:

- ресурсы одного сервера ограничены, и в какой-то момент наращивать их станет невозможно;
- топовое оборудование часто стоит непропорционально дороже, чем несколько менее мощных серверов с аналогичной суммарной производительностью.

3. Горизонтальное масштабирование (scaling out)

При горизонтальном масштабировании производительность увеличивается за счёт добавления в систему новых серверов (узлов) и распределения нагрузки между ними. Это позволяет преодолеть ограничения одного сервера.

Основные виды горизонтального масштабирования для баз данных:

- **Репликация** — создание полных копий базы данных на разных серверах для повышения отказоустойчивости.
- **Партиционирование** — логическое разделение данных внутри одного сервера.
- **Шардинг** — физическое разделение данных по разным серверам.

4. Партиционирование и шардинг

Партиционирование (Partitioning) — это разделение одной большой таблицы на несколько мелких частей (партиций) по определённому признаку, например по дате.

Важно, что все эти части хранятся на одном физическом сервере. Когда приложение запрашивает данные, СУБД обращается не ко всей огромной таблице, а только к нужной, маленькой партиции, что значительно ускоряет работу.

Шардинг (Sharding) — это следующий уровень оптимизации. Данные не просто делятся на части (шарды), а физически распределяются по разным серверам. Это даёт ещё больший прирост производительности, так как запросы обрабатываются ресурсами нескольких независимых машин.

Существует два вида шардинга:

- вертикальный
- горизонтальный

Вертикальный шардинг

В этом случае таблица делится по столбцам. Этот метод эффективен, когда таблица становится слишком «широкой», а разные группы столбцов используются для разных задач.

Пример: представим таблицу `users` с информацией о пользователях на одном сервере. Она содержит часто запрашиваемые данные (имя, email) и редко используемые, но объёмные или требующие особого доступа (например, биография, настройки уведомлений).

Таблица `users` на Сервере 1:

`id | username | email | password_hash | bio | settings`

users					
123	id				
A-Z	username				
A-Z	email				
A-Z	password_hash				
A-Z	bio				
A-Z	settings				

В процессе шардирования мы разделяем эту таблицу на две, размещая их на разных серверах.

- **На сервере 1 (Shard_Users_Profile)** остаётся таблица с основной, часто используемой информацией.

Таблица `users_profile`:

id	username	email
----	----------	-------

	users_profile
123	id
A-Z	username
A-Z	email

- **На сервере 2 (Shard_Users_Details)** создаётся таблица для дополнительной информации, связанная с основной через `user_id`.

Таблица `users_details`:

user_id	password_hash	bio	settings
---------	---------------	-----	----------

	users_details
123	id
A-Z	password_hash
A-Z	bio
A-Z	settings

Когда приложение запрашивает список пользователей для отображения, оно обращается только к лёгкой таблице на первом сервере. Для аутентификации или изменения профиля — ко второму. Это снижает нагрузку на каждый сервер и ускоряет целевые запросы. Однако логика объединения этих данных (если потребуется получить полный профиль пользователя) ложится на плечи приложения.

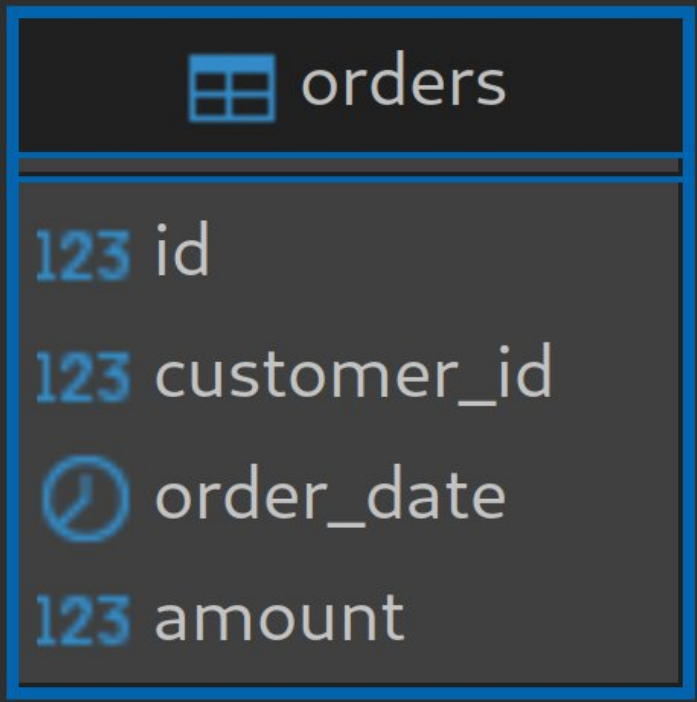
Горизонтальный шардинг

Это самый распространённый подход, при котором таблица делится по строкам. Он идеально подходит для таблиц с огромным количеством записей.

Пример: есть таблица `orders` с миллионами заказов на одном сервере. Поиск заказов по `customer_id` становится медленным из-за огромного размера таблицы и индекса.

Таблица `orders` на Сервере 1:

```
order_id | customer_id | order_date | amount
```



orders	
123	id
123	customer_id
🕒	order_date
123	amount

Процесс шардирования: мы выбираем ключ шардирования (в данном случае `customer_id`) и распределяем строки по разным серверам в зависимости от значения этого ключа.

- **Сервер 1 (Shard_1)** хранит заказы для клиентов с `customer_id` в диапазоне от 1 до 1 000 000.

Таблица `orders`:

`order_id | customer_id | order_date | amount` (только для `customer_id` 1–1M)

orders	
123	id
123	customer_id
🕒	order_date
123	amount

- **Сервер 2 (Shard_2)** хранит заказы для клиентов с `customer_id` в диапазоне от 1 000 001 до 2 000 000.

Таблица `orders`:

`order_id | customer_id | order_date | amount` (только для `customer_id` 1M+–2M)

orders	
123	id
123	customer_id
🕒	order_date
123	amount

Когда приложению нужно найти заказы клиента с `customer_id = 1 543 210`, оно уже знает, что нужно обращаться напрямую к Серверу 2. Запрос выполняется в разы быстрее, так как он работает с значительно меньшим объёмом данных.

4. Примеры реализации в Docker

Для практической реализации этих подходов удобно использовать Docker и Docker Compose. Docker позволяет быстро развернуть изолированные контейнеры с базами данных, а Docker Compose — управлять целой группой таких контейнеров (например, мастер-сервером и его шардами).

Партиционирование в PostgreSQL:

1. При создании основной таблицы указывается ключ, по которому она будет разделена: `PARTITION BY RANGE` (ключ).

Например:

```
create table avg_temp (  
  city_id uuid not null,  
  logdate date not null,  
  avg_temp float  
) partition by range (logdate);
```

2. Далее создаются таблицы-партиции, каждая для своего диапазона значений:

```
CREATE TABLE имя_партиции PARTITION OF основная_таблица FOR  
VALUES FROM (старт) TO (финиш)
```

```
create table avg_temp_jan25  
partition of avg_temp  
for values from ('2025-01-01') to ('2025-02-01');
```

```
create table avg_temp_feb25  
partition of avg_temp  
for values from ('2025-02-01') to ('2025-03-01');
```

```
create table avg_temp_mar25
partition of avg_temp
for values from ('2025-03-01') to ('2025-04-01');
```

3. После этого, при вставке данных в основную таблицу, PostgreSQL автоматически распределит их по нужным партициям.

```
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

insert into avg_temp (city_id, logdate, avg_temp)
values (uuid_generate_v4(), '2025-01-04', 1);

insert into avg_temp (city_id, logdate, avg_temp)
values (uuid_generate_v4(), '2025-02-03', 1);

insert into avg_temp (city_id, logdate, avg_temp)
values (uuid_generate_v4(), '2025-03-03', 1);
```

4. Проверка:

```
select * from avg_temp;
select * from avg_temp_jan25;
select * from avg_temp_feb25;
select * from avg_temp_mar25;
```

Шардинг в PostgreSQL

Для реализации шардинга используется Docker Compose, который позволяет управлять группой контейнеров: одним мастер-сервером и несколькими серверами-шардами. Вся логика распределения данных настраивается с помощью SQL на стороне PostgreSQL.

Рассмотрим пример с таблицей `books`, которую мы шардируем по `category_id`.

1. Архитектура в docker-compose.yml:

- Создаётся несколько сервисов: один мастер (master-db) и три шарда (shard1-db, shard2-db, shard3-db).
- Все контейнеры объединяются в одну Docker-сеть, что позволяет им обращаться друг к другу по именам сервисов.
- Каждому сервису через volumes подключается свой SQL-скрипт для первоначальной настройки.

```
... docker-compose.yml
1  version: '3.8'
2
3  services:
4    postgres_b:
5      image: postgres:17
6      container_name: "postgres_b"
7      env_file: .env
8      environment:
9        PGDATA: "/var/lib/postgresql/data/pgdata"
10     ports:
11       - "5632:5432"
12     volumes:
13       - ./conf/postgres_b/shards.sql:/docker-entrypoint-initdb.d/start.sql
14
15     postgres_b1:
16       image: postgres:17
17       container_name: "postgres_b1"
18       env_file: .env
19       environment:
20         PGDATA: "/var/lib/postgresql/data/pgdata"
21       ports:
22         - "5633:5432"
23       volumes:
24         - ./conf/postgres_b1/shards.sql:/docker-entrypoint-initdb.d/start.sql
25
26     postgres_b2:
27       image: postgres:17
28       container_name: "postgres_b2"
29       env_file: .env
30       environment:
31         PGDATA: "/var/lib/postgresql/data/pgdata"
32       ports:
33         - "5634:5432"
34       volumes:
35         - ./conf/postgres_b2/shards.sql:/docker-entrypoint-initdb.d/start.sql
36
37     postgres_b3:
38       image: postgres:17
39       container_name: "postgres_b3"
40       env_file: .env
41       environment:
42         PGDATA: "/var/lib/postgresql/data/pgdata"
43       ports:
44         - "5635:5432"
45       volumes:
46         - ./conf/postgres_b3/shards.sql:/docker-entrypoint-initdb.d/start.sql
47
48
```

2. Настройка мастер-сервера (скрипт master.sql):

- Включение расширения: `CREATE EXTENSION postgres_fdw;`

Это расширение (Foreign Data Wrapper) позволяет одному серверу PostgreSQL обращаться к таблицам, находящимся на других серверах, как будто они локальные.

- Регистрация шардов. Для каждого шарда создаётся «ссылка» на удалённый сервер, а также пользователь для подключения.

Пример для первого шарда:

```
CREATE SERVER books_1_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'postgres_b1', port '5432', dbname 'books');

CREATE USER MAPPING FOR "postgres"
  SERVER books_1_server
  OPTIONS (user 'postgres', password 'postgres');
```

- Создание «внешних таблиц». На мастере создаются таблицы, которые являются «прокси» к реальным таблицам на шардах:

```
CREATE FOREIGN TABLE books_1
(
  id bigint not null,
  category_id int not null,
  author character varying not null,
  title character varying not null,
  year int not null
) SERVER books_1_server
  OPTIONS (schema_name 'public', table_name 'books');
```

- Создание общего представления (View). Все внешние таблицы объединяются в одно представление, чтобы приложение могло работать с ними как с единой таблицей:

```
CREATE VIEW books AS
SELECT * FROM books_1
UNION ALL
SELECT * FROM books_2
UNION ALL
SELECT * FROM books_3;
```

- Настройка правил маршрутизации. Создаются правила, которые перехватывают команду INSERT в общее представление и перенаправляют данные на нужный шард в зависимости от значения category_id.

```
CREATE RULE books_insert_to_1 AS ON INSERT TO books
WHERE (category_id <= 3)
DO INSTEAD INSERT INTO books_1 VALUES (NEW.*);
```

3. Настройка серверов-шардов (скрипт shard.sql):

- На каждом шарде создаётся своя обычная таблица books.
- Добавляется ограничение CHECK, чтобы в таблицу конкретного шарда нельзя было по ошибке записать данные из чужого диапазона (например, CONSTRAINT category_id_check CHECK (category_id <= 3)).
- Для ускорения поиска по ключу шардирования создаётся индекс: CREATE INDEX on books (category_id);.

Пример для 1-го шарда:

```
CREATE TABLE books
(
    id bigint not null,
    category_id int not null,
    CONSTRAINT category_id_check CHECK (category_id <= 3),
    author character varying not null,
    title character varying not null,
    year int not null
);

CREATE INDEX books_category_id_idx ON books USING
btree(category_id);
```

После выполнения этих шагов система будет готова к работе: приложение сможет обращаться к общему представлению `books` на мастер-сервере, а данные будут автоматически распределяться по нужным шардам.